Características clase Animal POO

Enumerame cinco caracteristicas para un objeto perteneciente a la Clase Animal

Claro, pensando en **Programación Orientada a Objetos (POO)**, un objeto de la clase *Animal* puede tener varias características (atributos). Aquí tenés cinco típicas:

1. **Nombre** → Identifica al animal (ej: "Firulais")
2. **Edad** → Cantidad de años de vida
3. **Especie** → Tipo de animal (perro, gato, ave, etc.)
4. **Peso** → Peso corporal del animal
5. **Color** → Color predominante del animal

Si querés llevarlo a Java, serían atributos de la clase, por ejemplo:

Java

```java
public class Animal {
    String nombre;
    int edad;
    String especie;
    double peso;
    String color;
}
```

Si te interesa, después podemos avanzar a **métodos (comportamientos)** como `comer()`, `dormir()` o `hacerSonido()` para completar el modelo.

agreguemos esos tres métodos

Perfecto, completemos la clase agregando **comportamientos (métodos)** 👇

Java

```java
public class Animal {
    String nombre;
    int edad;
    String especie;
    double peso;
    String color;

    // Método: comer
    public void comer() {
        System.out.println(nombre + " está comiendo.");
```

```
    }

    // Método: dormir
    public void dormir() {
        System.out.println(nombre + " está durmiendo.");
    }

    // Método: hacer sonido
```

🔎 Mejora didáctica (muy útil para tus alumnos)

Podés explicar que:

- **Atributos → estado del objeto**
- **Métodos → comportamiento del objeto**

💡 Ejemplo de uso

Java

```java
public class Main {
    public static void main(String[] args) {
        Animal perro = new Animal();

        perro.nombre = "Firulais";
        perro.edad = 3;
        perro.especie = "Perro";
        perro.peso = 12.5;
        perro.color = "Marrón";

        perro.comer();
        perro.dormir();
        perro.hacerSonido();
    }
}
```

Si querés dar un paso más (ideal para clase), podemos:

- Sobrescribir `hacerSonido()` en subclases (`Perro`, `Gato`) 👉 **polimorfismo**
- O agregar **constructores y encapsulamiento (get/set)**

¿Seguimos con herencia? 🚀

modifiquemos el ejemplo para python

Perfecto 👍 llevemos el mismo modelo a **Python**, que es ideal para mostrar POO de forma más simple.

🐍 Clase `Animal` en Python

Python

```python
class Animal:
    def __init__(self, nombre, edad, especie, peso, color):
        self.nombre = nombre
        self.edad = edad
        self.especie = especie
        self.peso = peso
        self.color = color

    # Método: comer
    def comer(self):
        print(f"{self.nombre} está comiendo.")

    # Método: dormir
    def dormir(self):
        print(f"{self.nombre} está durmiendo.")

    # Método: hacer sonido
    def hacer_sonido(self):
```

💡 Ejemplo de uso

Python

```python
# Crear objeto
animal1 = Animal("Firulais", 3, "Perro", 12.5, "Marrón")

# Llamar métodos
animal1.comer()
animal1.dormir()
animal1.hacer_sonido()
```

🔎 Para explicar en clase

- `__init__` 👉 es el **constructor**
- `self` 👉 referencia al objeto actual
- No hace falta declarar tipos (Python es **dinámico**)

🚀 Siguiente paso didáctico (muy recomendable)

Podés mostrar **herencia y polimorfismo**:

Python

```python
class Perro(Animal):
    def hacer_sonido(self):
        print(f"{self.nombre} dice: ¡Guau!")
```

Python

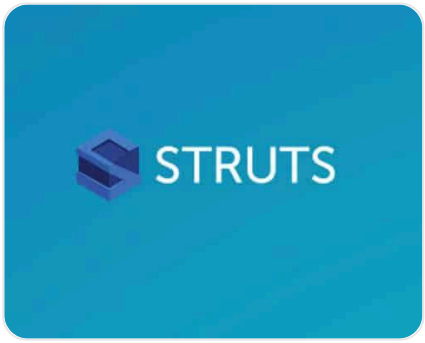
```python
perro1 = Perro("Firulais", 3, "Perro", 12.5, "Marrón")
perro1.hacer_sonido()
```

Si querés, te armo un **TP completo en Python** con consignas + solución para tus alumnos (muy en tu línea de clase).

Dame ejemplos de Frameworks en el desarrollo de Software para Java, Python, C# y JS

Perfecto, te dejo ejemplos claros por lenguaje (ideal para clase 👩‍🏫), con una breve descripción de cada framework:

☕ Java

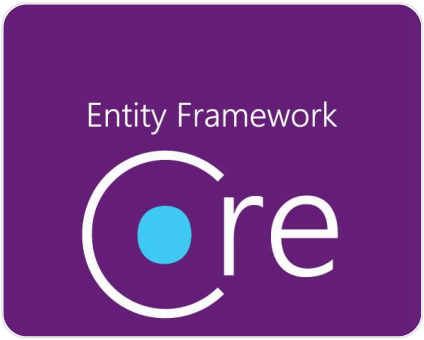


- Spring Boot → Framework moderno para crear aplicaciones web y APIs rápidamente (muy usado con microservicios).
- Hibernate → Framework ORM para mapear objetos Java a bases de datos.
- Apache Struts → Framework clásico basado en MVC (hoy menos usado).
- JavaServer Faces → Framework para construir interfaces web con componentes.

🐍 Python

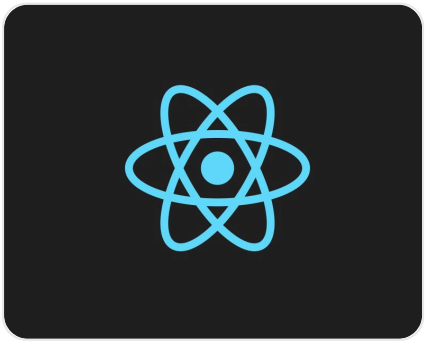
- **Django** → Framework completo (full-stack), incluye ORM, autenticación, admin, etc.
- **Flask** → Microframework liviano y flexible.
- **FastAPI** → Muy rápido, ideal para APIs modernas.
- **Pyramid** → Flexible, intermedio entre Flask y Django.

🔷 C#



- **ASP.NET Core** → Framework principal para desarrollo web en .NET.
- **Blazor** → Permite hacer frontend con C# en lugar de JavaScript.
- **Entity Framework** → ORM para bases de datos.
- **.NET Framework** → Plataforma clásica de desarrollo (ahora evolucionada a .NET).

🌐 JavaScript



- **React** → Biblioteca (muchos la consideran framework) para interfaces UI.

- **Angular** → Framework completo frontend.
- **Vue.js** → Framework progresivo y sencillo de aprender.
- **Express.js** → Framework backend para Node.js.

🎯 Resumen para tus alumnos

- **Framework = estructura + herramientas** para desarrollar más rápido
- Evitan "reinventar la rueda"
- Algunos son:
  - **Full-stack** → Django, Angular
  - **Microframeworks** → Flask, Express
  - **ORM** → Hibernate, Entity Framework

Si querés, te puedo armar esto en **PPT listo para clase + cuadro comparativo + TP práctico** 👍

Generemos encapsulamiento para el ejemplo en python

Perfecto 👌 ahora llevamos el ejemplo a **encapsulamiento en Python**, que es clave en POO para tus alumnos.

🔒 ¿Qué es encapsulamiento?

Consiste en **proteger los atributos** y acceder a ellos mediante métodos controlados (getters/setters).

En Python se usa la convención:

- `_atributo` → protegido (convención)
- `__atributo` → privado (name mangling)

🐍 Clase `Animal` con encapsulamiento

Python

```python
class Animal:
    def __init__(self, nombre, edad, especie, peso, color):
```

```python
        self.__nombre = nombre
        self.__edad = edad
        self.__especie = especie
        self.__peso = peso
        self.__color = color

    # Getters
    def get_nombre(self):
        return self.__nombre

    def get_edad(self):
        return self.__edad

    def get_especie(self):
        return self.__especie
```

💡 Ejemplo de uso

```python
animal1 = Animal("Firulais", 3, "Perro", 12.5, "Marrón")

# Acceso controlado
print(animal1.get_nombre())

animal1.set_edad(5)
animal1.set_peso(15)

animal1.comer()
```

🔎 Punto didáctico importante

Podés remarcar en clase:

- ❌ **Mal (rompe encapsulamiento):**

```python
animal1.__edad = -10
```

- ✅ **Bien (controlado):**

```python
animal1.set_edad(5)
```

🚀 Versión más “pythonica” (propiedades)

Si querés mostrar algo más moderno:

```python
class Animal:
    def __init__(self, nombre, edad):
        self.__nombre = nombre
        self.__edad = edad

    @property
    def edad(self):
        return self.__edad

    @edad.setter
    def edad(self, valor):
        if valor >= 0:
            self.__edad = valor
        else:
            print("Edad inválida")
```

Uso:

```python
animal = Animal("Firulais", 3)
animal.edad = 5   # parece atributo, pero valida
```

Si querés, el siguiente paso lógico para clase es:
👉 **Herencia + Polimorfismo + Encapsulamiento todo junto en un TP integrador** (te lo armo completo con consignas y solución docente).